CS260 - Written Assignment 2

Professor Galen Long, Professsor Kurt Schmidt, Professor Yusuf Osmanlioglu

Summer 2024-25

Total points: 100

- **For any algorithm design questions in this assignment and the course as a whole, give your algorithms in C/Python-like pseudocode *and* describe your algorithm briefly at a high level in plain English.**
- Your pseudocode can omit low-level details like memory allocations and type declarations.
- If you're not sure whether or not to include a detail in your pseudocode, err on the side of including it.
- Small mistakes like off-by-one errors will receive at most a small penalty. We know it's hard to write perfectly correct pseudocode because you can't execute it easily (although running your pseudocode on an example on paper is a great way to debug it).
- Whenever we ask you to design an algorithm, we expect your algorithm to be efficient (within reason). If an asymptotically more efficient algorithm exists, you may lose points if your algorithm is correct but asymptotically slower.
- However, you should prefer a correct but slow algorithm over an incorrect attempt at a more efficient algorithm.

1. Consider the following code:

```
function fooRec(L, n, i) {
  if (i == n) {
    print(L) // Assume this takes O(1) time
    return
  }
  L[i] = true
  fooRec(L, n, i + 1)
  L[i] = false
  fooRec(L, n, i + 1)
}

/* n is an integer */
function foo(n) {
  Create empty array L with capacity n // Assume this takes O(1) time
  fooRec(L, n, 0)
}
```

   (a) (5 points) Give a recurrence that represents the runtime of `foo`.

   (b) (10 points) Solve the recurrence using the substitution method.

   Hint: The geometric series states that $\sum_{i=0}^{k} x^i = \frac{1-x^{k+1}}{1-x}$. Be careful with your algebra. If a term is outside of parentheses, it's distributed to all terms inside the parentheses.

(c) (5 points) Give a tight bound on the runtime of `foo`.

2. (10 points) Given a list L with n unique positive integers all less than some constant k, give an algorithm that checks whether there exists two pairs of integers (p, q) and (r, t) where p, q, r, t in L, $p \neq q \neq r \neq t$, and $p + q = r + t$.

   Your solution should run in $O(n^2)$ time. Assume memory usage is not a concern. You only need to report if two such pairs exists, not what they were.

   Hint: If you use a hash table, you need to think about how you'd handle collisions, **if** you'd have any...

3. The following questions concern binary search trees (BSTs).

   (a) (5 points) Provide an example of two sequences of unique integer values, e.g. $A = (a_1, a_2, ..., a_n)$ and $A' = (a_1, a_3, a_{n-1}, ..., a_2)$, which contain the same elements in a different order and would each generate the same BST if inserted in their specified orders.

   (b) (15 points) A binary tree is *almost balanced* if, for every node, the total number of nodes in its left and right subtrees differs by at most one.

   Given a sorted array A of n unique integers, describe an algorithm that builds an almost balanced BST and runs in $\Theta(n)$ time.

   Briefly explain why your algorithm works correctly and analyze its runtime.

   (c) (10 points) Give an algorithm which converts any BST with n nodes into an almost balanced BST in $\Theta(n)$ time.

   Briefly explain why your algorithm works correctly and analyze its runtime.

4. (20 points) Give algorithms for how you would efficiently implement each of a priority queue's operations using **only** one or more stack(s), and analyze the runtime of each operation. Assume all priorities will be integers and you'll only need to store priorities, not values. Recall that a priority queue supports these operations:

   1. *insert*(*priority*): Inserts *priority* into priority queue
   2. $extractMin()$: Removes and returns min *priority* from priority queue

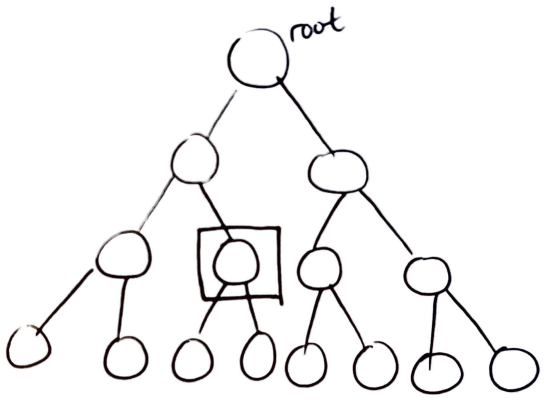
   And a stack supports these operations:

   1. *push*(x): Inserts x into stack
   2. *pop*(): Removes and returns most recently pushed element
   3. $isEmpty()$: Returns true if stack empty, else false

   Remember that you are **not** allowed to access elements at arbitrary indices in a stack; you can only pop the "top" element.

   Hint: You may use more than one stack. Remember that $extractMin$ should return *and* remove the min priority. Consider how you should handle duplicate priorities.

5. (20 points) Say you have a heap represented as a recursive node structure, where you are only given the root note, and each node has a reference to its left child, right child, and parent. The nodes are not stored in an array, so the only way to access other nodes is by traversing pointers starting at the root node, e.g. $root.left.right$ would access this node:



Given only the root node of a heap represented as a recursive node structure of n nodes, n, and a number x, give an efficient algorithm to insert x into the heap and analyze its runtime. You don't need to redescribe how upheap works.

Hint: It's possible to do better than linear time.